AMLNN Advanced Quantization User Guide

版本: 0.1

发布日期: 2026-09-24

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称"Amlogic"）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址：www.amlogic.cn
- 售前咨询：contact@amlogic.com
- 技术支持：support@amlogic.com

修订记录

版本	变更描述
V0.1 (2026-09-24)	初始版本。

目录

1 文档说明
- 1.1 文档目的
- 1.2 支持平台
- 1.3 关键字说明

2 量化介绍
- 2.1 量化简介
  - 2.1.1量化定义与映射区间
  - 2.1.2 量化参数计算公式
- 2.2 量化能力介绍
- 2.3 量化算法介绍
  - 2.3.1 Feature量化算法
  - 2.3.2 Weights 量化算法
  - 2.3.3 Smooth O3量化算法
- 2.4 量化策略
  - 2.4.1 CV 模型量化指导
  - 2.4.2 Transformer 模型量化指导
  - 2.4.3 量化指导总结

3 量化转换
- 3.1 量化转换示例
- 3.2 模型格式与量化算法适配说明
- 3.3 不同量化粒度下的模型精度测试

4 混合量化指导
- 4.1 混合量化简介
- 4.2 混合量化指导

5 quantize_optimization_tool 使用指导
- 5.1 工具介绍
- 5.2 使用指导
- 5.3 使用建议
- 5.4 自定义评估标准

6 hybrid_ratio_tool使用指导
- 6.1 工具介绍
- 6.2 YOLOX-S 模型验证
  - 6.2.1 W4A16 量化验证
  - 6.2.2 w4a8量化验证
- 6.3 使用建议

1 文档说明

1.1 文档目的

本文档旨在为 Amlogic NPU 平台提供量化转换的进阶指南，系统阐述当前支持的各类量化格式（Quantization Format）在权重（Weights）与特征图（Feature Maps）上对应的量化算法。同时，重点介绍 `hybrid_ratio_tool` 与 `quantize_optimization_tool` 两款量化搜索工具，指导开发者借助工具自动化搜索，获取最佳量化精度与性能的平衡方案。

1.2 支持平台

本指南适用于以下 Amlogic NPU 平台：

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

> C305X2/A123X 和 A311Y3 属于**ADLA3**，其他平台属于**ADLA2**.

1.3 关键字说明

- **ADLA2（第二代Amlogic NPU芯片）**：硬件指令集仅支持int8、uint8、int16等定点整数格式运算，无原生浮点加速单元。
- **ADLA3（第三代Amlogic NPU芯片）**：在定点运算能力基础上新增原生硬件支持，支持fp16、bf16浮点格式与int4低比特格式的运算。
- **Feature量化（特征图量化）：**针对推理过程中生成的特征图激活值执行的量化操作，默认采用逐层（Perlayer）量化粒度。
- **Weights量化（权重量化）**：针对模型中可训练权重参数执行的量化操作。
- **Perchannel量化（逐通道量化）：** 以单个输出通道为独立单位进行量化，通常针对卷积层、全连接层的权重启用。
- **Perlayer量化（逐层量化）：** 以完整张量所属的单层网络为整体单位进行统一量化。
- **Pergroup量化（子通道分组量化）：** 将单通道内的权重进一步划分为多个独立分组，以子通道组为单位执行量化，在更低比特量化场景下可显著降低精度损失。

2 量化介绍

2.1 量化简介

模型量化是一种高效的模型压缩与部署优化技术，将原本以32bit/64bit高精度浮点数存储的模型参数，转换为16bit、8bit甚至更低的4bit低精度格式存储，在可接受的轻微精度损失下，实现模型体积、内存占用的倍数级缩减，推理速度大幅提升。

2.1.1量化定义与映射区间

量化的核心是将高精度的浮点数据映射到低精度的整数区间，不同位宽对应的映射范围如下：

量化类型	数据类型	映射区间 (Quantized Range)	说明
UInt8	Unsigned Int8	[0,255]	无符号8位整数
Int8	Signed Int8	[−128,127]	有符号8位整数
Int16	Signed Int16	[−32768,32767]	有符号16位整数，用于对精度要求较高的场景

2.1.2 量化参数计算公式

量化过程主要涉及两个关键参数：**Scale（缩放比例）**与 **Zero Point（零点偏移）**。

- **UInt8 / Int8 量化公式（非对称量化）**

  适用于数据分布不对称或包含负数的场景，通过 Zero Point 灵活调整零点位置。

  - **Scale 计算**：
    `scale=(float_max−float_min)/(Qmax−Qmin)`
  - **Zero Point 计算**：
    `zero_point=Qmax−float_max/scale`
    注：计算结果通常需四舍五入取整，并截断至对应整数类型的合法取值范围内。

> **符号说明**：
>
> - `float_max / float_min`：浮点数据的最大值 / 最小值
> - `Qmax / Qmin`：量化目标类型的最大 / 最小整数值（如 Int8 为 127 / −128）

- **Int16 量化公式（对称量化）**

  通常采用对称量化策略，强制零点为 0，简化计算逻辑，适合动态范围较大且分布相对对称的数据。

  - **中间变量计算**：
    `float_max_temp = max(|float_max|, |float_min|)`
  - **Scale 计算**：
    `scale = float_max_temp / Qmax`（其中 `Qmax = 32767`）
  - **Zero Point**：
    `zero_point=0`

2.2 量化能力介绍

Amlogic NPU 全平台量化能力对照表：

平台型号	Feature（特征图）量化	Weights（权重）量化	专属高阶能力
ADLA2	✅ int8 定点量化 ✅ int16 定点量化	✅ int8 定点量化	• 仅支持基础定点量化
ADLA3	✅ int8 定点量化 ✅ int16 定点量化 ✅ fp16 / bf16 非量化模式	✅ int8 定点量化 ✅ int4 低比特量化 ✅ fp16 / bf16 非量化模式	• 支持基础定点量化 • 支持低比特大模型部署、高精度非量化推理

> **特殊说明**：
>
> - 部分硬件型号未直接对应A311Y3、C305X2、A123X等公开命名ADLA3平台，需参考<02_Amlogic_NPU_User_Guide>文档的 **2.1.3章节：平台类型确认**，通过`target_platform`字段完成能力映射对齐，确保量化配置与硬件能力完全匹配，避免出现算子不兼容、精度异常等问题。

量化粒度支持能力对照表：

量化对象	数据精度	支持粒度	适用算子/说明
Feature (激活值)	INT8 / INT16	perlayer	全平台统一支持逐层量化，即同一层内所有通道共享同一套量化参数。
Weights (权重)	INT8	perlayer perchannel	• Per-layer：全层共享量化参数。 • Per-channel：按输出通道维度独立量化，显著提升 Conv/FC 算子精度。
Weights (权重)	INT4	perchannel pergroup	• Per-channel：按输出通道维度独立量化。 • Per-group：仅针对 FC (全连接) 算子支持分组量化，进一步降低大模型权重显存占用。

> 详细说明：
>
> - **Feature 量化限制**：目前特征图（Activation）量化仅支持 **perlayer** 模式
> - **Weights INT8 量化**：支持 **perlayer** 和 **perchannel** 两种模式
> - **Weights INT4 低比特量化**：
>   - **Conv / DwConv 算子**：4bit 量化采用 **perchannel（逐通道）** 方式
>   - **FC 算子**：4bit 量化采用 **per-group（逐分组）** 方式，分组粒度由 group_size 参数配置。

2.3 量化算法介绍

2.3.1 Feature量化算法

- **normal（Min-Max 量化，默认）**
  - **原理**：直接选取校准数据集中的全局最大值与最小值作为量化边界。
  - **特点**：计算逻辑最简单、开销最低。
  - **适用场景**：数据分布均匀、无明显离群值（Outliers）的场景。
- **percentile（百分位数量化）**
  - **原理**：为避免全局极值受离群点干扰，采用高分位数（如 99.9%）和低分位数（如 0.1%）截取有效数值范围。结合指数滑动平均（EMA）对多轮校准结果进行平滑处理。
  - **特点**：能有效抑制极端异常值对量化区间的影响，提升量化稳定性。
  - **适用场景**：激活值存在少量极端离群点，但整体分布相对集中的场景。
- **histogram（直方图量化）**
  - **原理**：首先统计激活值的绝对值直方图以刻画数据分布特征；随后在候选阈值空间中遍历，计算量化前后分布差异（如 KL 散度），选取差异最小的单轮最优阈值；最后结合 EMA 多轮平滑得到最终边界。
  - **特点**：基于数据分布相似性优化阈值，能更好地保留原始数据的统计特性。
  - **适用场景**：数据分布复杂、非均匀，且对分布一致性要求较高的激活值量化场景（类似 TensorRT 中的 KLD 策略）。
- **omse（Optimal Mean Squared Error，最优均方误差）**
  - **原理**：一种基于搜索的优化方案。在原始数据范围 `[raw_min, raw_max]` 基础上，不断向内收缩候选区间 `[new_min, new_max]`，遍历多组候选区间并计算量化前后的整体均方误差（MSE），最终选择 MSE 最小的区间作为量化边界。
  - **特点**：以最小化重构误差为目标，通常能获得更高的数值还原精度，但计算开销相对较大。
  - **适用场景**：对量化精度极其敏感、允许较高校准计算成本的场景。

2.3.2 Weights 量化算法

- **normal（Min-Max 量化，默认）**
  - **原理**：直接选取校准数据集中的全局最大值与最小值作为量化边界。
  - **特点**：计算逻辑最简单、开销最低。
  - **适用场景**：数据分布均匀、无明显离群值（Outliers）的场景，如模型权重量化。
- **lsq (Learned Step Size Quantization)**
  - **原理**：基于梯度反向传播的量化参数学习算法。将量化步长（Scale）视为可训练参数，利用直通估计器（STE）解决取整操作的梯度中断问题，并在反向传播中引入针对比特位宽的梯度缩放因子以平衡更新幅度。在量化转换阶段，通常采用逐层优化策略，固定其他层参数，单独微调当前层的权重与量化步长。
  - **特点**：通过端到端的梯度下降寻找最优量化参数，能显著降低极低比特（如 W4A8）场景下的量化误差，保持模型精度。
  - **适用场景**：对量化精度要求极高、且允许一定校准计算成本的极低比特权重量化场景。
- **decoupleQ**
  - **原理**：基于数学优化视角的权重量化解耦算法。摒弃传统对离群值的启发式处理，将量化建模为带约束的优化问题。核心机制是将模型权重解耦为“整数部分”与“浮点残差部分”，通过交替优化方法分别求解最优的整数量化值与浮点补偿参数。
  - **特点**：生成的量化权重具有线性均匀（Uniform）特性，更适配硬件加速引擎；在不引入额外推理算子开销的前提下，实现接近 FP16/BF16 原始精度的恢复效果。
  - **适用场景**：低比特（如4-bit）权重量化场景，特别是需要高硬件效率与高精度平衡的大模型部署

2.3.3 Smooth O3量化算法

Smooth O3 是基于经典 SmoothQuant 理论，针对 Amlogic NPU 硬件架构深度优化的进阶训练后量化（PTQ）方案。其核心目标是解决大模型及复杂 CV 模型中因激活值存在极端离群点（Outliers）而导致的量化精度崩塌问题，从而在 Conv2d、Depthwise Conv2d 及 FC 等关键算子上实现高精度的定点量化（如 W8A8）

1. 核心原理与功能机制

Large Language Models (LLMs) 和深层 Transformer 结构的激活值分布往往极不均匀，存在少量数值极大的离群点。直接量化会迫使量化区间过大，导致大部分普通数值的表示精度严重损失。Smooth O3 通过"难度迁移"策略解决此问题：

- **激活值平滑**：对输入激活值进行分通道除以平滑因子（Smoothing Factor），大幅压缩离群点的幅度，使激活值分布更加均匀，更适配 INT8/INT16 的有限表示范围。
- **权重等效补偿**：为了保持数学等价性，将相同的平滑因子乘入权重矩阵。由于权重分布本身较为平滑且冗余度高，这种变换不会显著增加权重的量化误差。
- **结果**：将量化的主要难点从"难以量化的激活值"转移到了"易于量化的权重"上，从而在整体上将量化误差降至最低。

2. 图结构变更

算子插入：平滑因子不会被硬编码或忽略，而是以独立的 **Mul（乘法）算子**形式，自动插入到目标算子（Conv2d/FC等）的输入前端。

3. 使用指南

- **配置方法：**在量化转换脚本的配置接口中，设置以下参数即可全局启用：

```
amlnn.config(apply_smooth_o3 = True)
```

- **推荐启用场景**
  - **全定点量化（W8A8）**：建议开启。这是 Smooth O3 收益最大的场景，部分模型能显著提升 INT8 权重与 INT8 激活值配合下的模型精度。
  - **混合精度量化（W8A16）**：建议开启。虽然激活值保留较高精度，但平滑处理仍能优化权重分布，进一步提升整体鲁棒性。
- **实测收益说明**
  - **最优解标配**：目前内部验证过的大多数w8a8量化方案，均开启了 Smooth O3 能力。

2.4 量化策略

针对 CV 模型与 Transformer 模型在量化策略上的显著差异，本章节介绍如何选择合适的量化指导。

为精准评估不同量化策略（如 INT8 Per-channel、混合精度等）对模型精度的实际影响，amlnn-toolkit 内置了基于标准格式的快速验证机制：只需在配置接口中设置 `export_intermediate=True`，即可在量化转换流程中直接导出已完成量化参数注入的中间 TFLite 模型。基于该模型，您可直接通过 TFLite Runtime 执行推理测试，快速测算量化前后的误差差异，高效定位精度瓶颈。

2.4.1 CV 模型量化指导

对于传统的卷积神经网络（如 ResNet, YOLO, MobileNet等），其激活值分布相对均匀，对量化误差的容忍度较高。推荐"递进式"全整数量化，建议采用**从低精度到高精度、从粗粒度到细粒度的递进式调优路径**，优先追求极致的推理速度与内存压缩。

推荐调优路径：

1. **baseline（基准）：** 采用INT8逐层量化，搭配MinMax校准作为初始方案

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
       quantized_dtype='w8a8', quantized_method='perlayer', \
       target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

   - ***预期*：** 大多数经典 CV 模型在此配置下即可保持极高精度，且推理速度最快。
2. **Step 1（精度微调）**：若精度不足，将权重量化粒度升级为INT8逐通道，保留MinMax校准。
   - **原理：** 卷积核不同通道的数值分布差异较大，perchannel 能显著减少权重量化误差。

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a8', quantized_method='perchannel',
apply_smooth_o3=True,\
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

3. **Step 2（激活值优化）**：精度仍不达标时，激活值校准切换为percentile或histogram或omse，权重校准改用lsq或decoupleQ(下一个工具版本开始支持)。
   - ***原理*：** CV 模型激活值中可能存在少量离群点，百分位或直方图算法能更鲁棒地确定量化边界。
   - 可参考文档第 5 章节：quantize_optimization_tool 使用指导，借助该工具快速遍历校准算法、量化粒度等关键参数组合，搜索W8A8 量化的精度最优解。

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a8', quantized_method='perchannel',
apply_smooth_o3=True, \
    activation_quant_algo="histogram",weight_quant_algo="lsq", \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

4. **Step 3（激活值升级）**：若上述 INT8 优化手段均失效，考虑将激活值升级为 `INT16`。

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a16', quantized_method='perchannel',
 activation_dtype='i16', \
    activation_quant_algo="histogram",weight_quant_algo="lsq", \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

5. **Step 4 (激活不量化)**: 精度仍不足时，将激活值切换为 f16/bf16 不量化模式 **（仅ADLA3支持）**

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a16', quantized_method='perchannel',
 activation_dtype='f16', \
    activation_quant_algo="histogram", weight_quant_algo="lsq", \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
#or activation_dtype='bf16'
```

6. **Step5 (全量非量化)**: 若仍不满足要求，权重与激活值统一切换为 f16/bf16 非量化模式 **（仅ADLA3支持）**

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w16a16',activation_dtype='f16', \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
 #or activation_dtype='bf16'
```

> **核心逻辑**：CV 模型结构简单、局部性强，通常不需要混合精度，通过精细化的 PTQ（训练后量化）参数调整即可实现 W8A8 全 INT8 部署。
>
> **ADLA2适配说明**：Step4、Step5仅支持ADLA3平台。若在ADLA2上执行到Step3后精度仍不达标，需采用INT16+Float混合量化方案，其中Float算子运行在CPU上。

2.4.2 Transformer 模型量化指导

LLM、ViT、Swin Transformer等Transformer类架构依赖全局注意力机制，激活值分布呈现显著的重尾特性，存在大量极端离群值，同时LayerNorm、Softmax等关键算子对量化误差的容忍度极低。若直接套用传统CNN的全INT8/INT16整数量化方案，极易出现精度断崖式崩塌。ADLA2平台推荐采用**混合精度量化**策略，浮点算子自动调度至CPU运行；ADLA3平台可直接启用原生FP16/BF16硬件加速特性。

推荐路径：

ADLA2：

在 ADLA2 平台部署 Transformer 模型时，直接采用全 INT8/INT16 量化，大多数模型的精度指标无法达标，因此混合精度量化(INT16+Float混合)是该架构下的必选方案。

ADLA3：

- **Step 1 (激活不量化 w4a16)**: 激活使用f16/bf16非量化模型，权重使用4bit。

```
amlnn.config(quantized_dtype='w4a16', quantized_method='perchannel',
 activation_dtype='f16', \
    weight_quant_algo="decoupleQ", target_platform='PRODUCT_PID0XA007',
export_intermediate=True)
#or activation_dtype='bf16'
```

- **Step 2 (激活不量化 w8a16)**: 激活使用f16/bf16非量化模式，权重使用8bit。

```
amlnn.config(quantized_dtype='w8a16', quantized_method='perchannel',
 activation_dtype='f16', \
    weight_quant_algo="lsq", target_platform='PRODUCT_PID0XA007',
export_intermediate=True)
#or activation_dtype='bf16'
```

- **Step3 (全量非量化)**: 权重与激活值统一切换为 f16/bf16 非量化模式

```
amlnn.config(quantized_dtype='w16a16', activation_dtype='f16', \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
#or activation_dtype='bf16'
```

2.4.3 量化指导总结

特性	CV 模型 (CNN)	Transformer 模型 (ViT/LLM)
数据分布	相对均匀，局部相关性强	极不均匀，存在严重离群值 (Outliers)
敏感算子	较少，Conv/FC 较鲁棒	多，LayerNorm, Softmax, Attention 极度敏感
首选策略	全 INT8 量化 (W8A8)	• **ADLA2:** 混合精度 (Mixed-Precision) • **ADLA3:** 使用激活不量化、或者激活权重均不量化方案
调优重点	量化粒度 (Per-channel) 与校准算法	层敏感度分析、关键层精度保留
推荐路径	INT8 Per-layer → Per-channel → 高级校准算法	w4a16 --> w8a16 --> w16a16
备选方案	INT16 (仅当 INT8 彻底失败时)	w4a16/w8a16多种混合

3 量化转换

3.1 量化转换示例

```
import os
from amlnn.api import AMLNN

def convert_and_quantize_model():
    """
    使用 amlnn-toolkit 进行 ONNX 模型加载、量化配置、编译及导出 ADLA 模型。
    """
    # 1. 初始化 AMLNN 实例
    # log_level: 设置日志级别，INFO 用于查看常规流程，DEBUG 用于排查细节
    amlnn = AMLNN(log_level='INFO')

    # 2. 定义路径与基础配置
    model_path = 'resnet34-v1-7.onnx'
    dataset_path = './dataset.txt'
    output_dir = 'resnet34-v1-7_adla_unify'

    try:
        # 3. 加载 ONNX 模型
        print(f"[Step 1] Loading ONNX model: {model_path}")
        amlnn.load_onnx(model=model_path)

        # 4. 配置量化与编译参数
        print("[Step 2] Configuring quantization parameters...")

        # 量化核心参数说明：
        # - quantized_dtype: 'w8a8' 表示权重和激活值均使用 INT8 量化，兼顾精度与速度
        # - quantized_method: 'perchannel' 逐通道量化，通常比 per-tensor 精度更高
        # - apply_smooth_o3: 启用 SmoothQuant 等优化技术，减少离群值对量化的影响
        # - normalization: 根据模型训练时的预处理方式设置 Mean/Std
        amlnn.config(
            normalization_mean=[[128, 128, 128]],
            normalization_std=[[128, 128, 128]],
            quantized_dtype='w8a8',           # 量化类型：INT8 权重 + INT8 激活
            quantized_method='perchannel',    # 量化粒度：逐通道
            apply_smooth_o3=True,             # 启用高级平滑优化
            target_platform='PRODUCT_PID0XA007', # 目标硬件平台 ID
            export_intermediate=True          # 导出中间层模型，便于确认量化精度
        )

        # 5. 模型编译与量化校准
        # dataset: 提供代表性数据集用于 PTQ (Post-Training Quantization) 校准
        print(f"[Step 3] Compiling and calibrating with dataset:
{dataset_path}")
        amlnn.compile(dataset=dataset_path, iterations=200)

        # 6. 导出 ADLA 模型
        print(f"[Step 4] Exporting ADLA model to directory: {output_dir}")
        amlnn.export_adla()
```

```
        print("[Success] Model conversion completed successfully.")

    except Exception as e:
        print(f"[Error] Failed to convert model: {str(e)}")
        raise

if __name__ == '__main__':
    convert_and_quantize_model()
```

> **量化配置与数据准备指南**
>
> - **参数配置策略**：量化核心参数（如量化类型、粒度、量化算法、平滑策略等）需通过 `config()` 接口进行定义；而校准所需的动态范围统计则依赖于 `compile()` 接口执行
> - **校准数据集要求**：请确保 `./dataset.txt` 指向的文件中包含**具有代表性**的输入样本
>   - **数量建议**：通常 **100~1000** 张图像或者npy数据集即可满足 PTQ（训练后量化）的校准需求。
>   - **质量建议**：数据分布应覆盖业务场景中的典型情况，以避免因数据偏差导致的精度损失。

3.2 模型格式与量化算法适配说明

本章节主要介绍不同硬件平台适配的模型存储格式，以及各类格式转换过程中支持的特征（Feature）与权重（Weights）量化算法。更完整的量化转换参数说明，请参考文档《03_Amlogic_NPU_API_Reference_AMLNN》。

- **不同硬件平台适配的模型存储格式支持情况汇总如下：**
  - **Support Params**：定义对应 Format 所需的配置参数。

ADLA	target_platform	Format(weights*Feature)	Support Params
ADLA2	PRODUCT_PID0XA001 ~ PRODUCT_PID0XA006	Int8*Int8	quantized_dtype='w8a8'
		Int8*Int16	quantized_dtype='w8a16', activation_dtype='i16'
ADLA3	PRODUCT_PID0XA007/PRODUCT_PID0XA008	Int4*int8	quantized_dtype='w4a8'
		Int8*Int8	quantized_dtype='w8a8'
		Int4*f16	quantized_dtype='w4a16' , activation_dtype='f16'
		Int4*bf16	quantized_dtype='w4a16' , activation_dtype='bf16'
		Int8*int16	quantized_dtype='w8a16', activation_dtype='i16'
		Int8*f16	quantized_dtype='w8a16', activation_dtype='f16'
		Int8*bf16	quantized_dtype='w8a16', activation_dtype='bf16'
		f16*f16	quantized_dtype='w16a16', activation_dtype='f16'
		bf16*bf16	quantized_dtype='w16a16', activation_dtype='bf16'

- **各格式对应的 Feature 及 Weight 量化算法汇总：**

Format (weights*Feature)	quantized_dtype	activation_dtype	weight_quant_algo	activation_quant_algo

Format (weights*Feature)	quantized_dtype	activation_dtype	weight_quant_algo	activation_quant_algo
Int4*int8	w4a8	None	normal/decoupleQ/lsq	normal/histogram/omse/percentile
Int8*int8	w8a8	None	normal/lsq	normal/histogram/omse/percentile
Int4*f16	w4a16	f16	normal/decoupleQ/lsq	/
Int4*bf16	w4a16	bf16	normal/decoupleQ/lsq	/
Int8*int16	w8a16	i16	normal/lsq	normal/histogram/omse/percentile
Int8*f16	w8a16	f16	normal/lsq	/
Int8*bf16	w8a16	bf16	normal/lsq	/
f16*f16	w16a16	f16	/	/
bf16*bf16	w16a16	bf16	/	/

- **config接口参数说明：**
  - **activation_quant_algo：激活值量化算法，有效值[normal，histogram，percentile，omse]**
    - **histogram量化算法参数**
      - histogram_bins：直方图校准算法的核心配置参数。对浮点激活值取绝对值后，按指定的 bins 数量进行区间分桶，遍历计算不同截断区间对应的量化误差，最终选取误差最小的区间，输出最优的量化范围max_value与min_value。**默认值:2048**，有效值：[1024,2048,4096]
    - **percentile量化算法参数**
      - percentile_alpha：百分位截断阈值，默认0.99999，有效值：(0~1)。直接指定量化上分位点，自动以`1-percentile_alpha`作为下分位点，截断两端极少量离群值，避免量化区间被极端值过度拉大。数值越大，截断范围越小，量化区间越贴近原始数据分布。
      - percentile_sigma：指数移动平均平滑系数，默认0.01，有效值：(0~1)。用于多批次校准场景下动态更新统计量，公式为`val_new = (1-sigma)*val_old + sigma*val_current`。数值越小，历史数据权重越高，单批次新数据占比越低，统计结果抗波动能力越强。
    - **omse量化算法参数**
      - omse_range：定义 OMSE（Optimal Mean Square Error）算法生成的候选截断区间总数。算法会基于该数量生成多组具有不同收缩程度的候选区间，逐一计算每组区间的量化均方误差（MSE），最终选取误差最小的一组作为最优量化截断范围。数值越大，搜索空间越精细，但计算开销相应增加。**默认值：90**，有效值：(0~100)
      - omse_step：定义每轮迭代中量化区间上下边界同步向内收缩的比例增量。在遍历候选区间的过程中，每迭代一轮，收缩系数会在上一轮基础上累加一个`omse_step`值，从而逐步缩小量化范围以寻找最小 MSE 对应的最佳截断点。该参数控制了区间收缩的粒度与搜索效率。**默认值:0.02**，有效值：(0~1)
  - **weight_quant_algo：权重量化算法，有效值[normal/decoupleQ/lsq]**
  - **apply_smooth_o3**：Smooth O3平滑优化开关，默认值为`false`。
  - **quantized_method**：权重量化粒度，支持 perchannel / perlayer，默认 perchannel。perchannel 按通道分配量化参数，精度更高；perlayer 整层共享一套参数，部分模型易出现明显掉点。
  - **inference_input_type**：指定模型导出时的输入数据类型，默认值为`"quant"`（即与量化类型保持一致）。

    该参数需根据`quantized_dtype`和`activation_dtype`的配置选择合法值：

        - 当 `quantized_dtype` 为 `'w4a8'` 或 `'w8a8'` 时，支持设置为 `'int8'`、`'uint8'` 或 `'float32'`。
        - 当 `quantized_dtype` 为 `'w8a16'` 且 `activation_dtype` 为 `'i16'` 时，支持设置为 `'int16'` 或 `'float32'`。
    - **inference_output_type**：指定ADLA模型导出的输出数据类型，默认值为 `"quant"`（与量化转换类型保持一致）。
        - 当 `quantized_dtype` 为 `'w4a8'` / `'w8a8'` 时，支持配置为 `int8`、`uint8`、`float32`。
        - 当 `quantized_dtype` 为 `'w8a16'` 且 `activation_dtype` 为 `'i16'` 时，支持配置为 `int16`、`float32`。
    - **export_intermediate**：控制是否将转换过程中的中间 IR（中间表示）导出为 TFLite 模型文件，用于验证量化精度。默认值为 `False`不导出。
    - **group_size**：FC 算子 4-bit 量化的分组粒度，仅当 `quantized_dtype` 为 `w4a8` 或 `w4a16` 时生效。支持 32 / 64 / 128。数值越小精度越高，数值越大推理效率越高。
    - **hybrid_ratio**：仅在 `w4a8`、`w4a16` 量化模式下生效，控制卷积算子 4-bit 权重量化的占比。数值越大，量化压缩比例越高、显存占用越低，精度损失风险也相应增加。
- **compile接口参数说明：**
    - **dataset**：量化校准数据集配置文件路径。文件内需列出用于统计激活值分布的图片或 `.npy` 数据路径。
        - **默认值**：`None`（使用随机数生成虚拟数据进行量化）。
        - **注意**：默认模式仅适用于验证转换流程或测试推理性能，因缺乏真实数据分布统计，**无法保证模型精度**。正式部署请务必提供具有代表性的真实校准数据。
    - **iterations**：量化校准使用的样本数量，从 `dataset.txt` 指定的数据集中读取。
        - **默认值**：`-1`（遍历数据集中的全部样本）。
        - **特殊行为**：若未配置数据集（`dataset` 为 `None`），则忽略此参数，仅生成单组随机数据进行量化（仅用于流程测试，无实际精度意义）。

3.3 不同量化粒度下的模型精度测试

所有模型采用 **w8a8 + perchannel + normal 校准 + normal 权重量化** 的基准方案时，Top1精度损失均控制在 **3个百分点以内**；其中 ResNet50、ViT-B/16、MobileNetV2 表现更优，基准精度损失低于 **1.5个百分点。**

在此基础上，通过组合激活校准算法、LSQ 权重量化、SmoothO3 平滑优化策略，所有模型的最优量化方案精度损失进一步压缩至 **1.5个百分点以内**；MNASNet1_3 仅损失 0.4 个百分点，ResNet50 甚至实现量化后精度反超浮点基准，整体量化优化收益显著

模型名称	SmoothO3	量化粒度	特征校准算法	权重量化算法	量化后Top1精度	浮点基准Top1精度	精度损失率	备注
EfficientNet-B7	❌	perchannel	normal	normal	0.7145	0.7300	-1.55%	基础量化方案
	❌	perchannel	percentile	lsq	**0.7175**	0.7300	-1.25%	精度最优配置
	✅	perchannel	percentile	normal	0.7175	0.7300	-1.25%	开启平滑优化

模型名称	SmoothO3	量化粒度	特征校准算法	权重量化算法	量化后Top1精度	浮点基准Top1精度	精度损失率	备注
MNASNet1_3	❌	perchannel	normal	normal	0.7395	0.7590	-1.95%	基础量化方案
	❌	perchannel	percentile	lsq	**0.7550**	0.7590	-0.40%	精度最优配置
MobileNetV2	❌	perchannel	normal	normal	0.7080	0.7215	-1.35%	基础量化方案
	❌	perchannel	percentile	normal	**0.7150**	0.7215	-0.65%	精度最优配置
RegNetX-16GF	❌	perchannel	normal	normal	0.7605	0.7900	-2.95%	基础量化方案
	❌	perchannel	histogram	lsq	**0.7760**	0.7900	-1.40%	精度最优配置
	✅	perchannel	histogram	normal	0.7760	0.7900	-1.40%	开启平滑优化
RegNetY-16GF	❌	perchannel	normal	normal	0.7650	0.7900	-2.50%	基础量化方案
	❌	perchannel	histogram	lsq	**0.7770**	0.7900	-1.30%	精度最优配置
ResNet50	❌	perchannel	normal	normal	0.7950	0.7995	-0.45%	基础量化方案
	❌	perchannel	histogram	lsq	**0.8000**	0.7995	+0.05%	量化后精度反超浮点
ShuffleNetV2	❌	perchannel	normal	normal	0.7345	0.7600	-2.55%	基础量化方案
	❌	perchannel	percentile	lsq	**0.7480**	0.7600	-1.20%	精度最优配置
ViT-B/16	❌	perchannel	normal	normal	0.7845	0.7980	-1.35%	基础量化方案
	❌	perchannel	percentile	normal	**0.7860**	0.7980	-1.20%	精度最优配置

perlayer 粒度下的 normal+normal 基准量化，精度损失普遍远高于 perchannel 方案，多数模型损失超过 **3个百分点**，其中 ShuffleNetV2、ViT-B/16 的基准损失突破 **13个百分点**，逐层量化的基础精度短板十分突出。

通过组合特征校准算法、LSQ 权重量化、SmoothO3 平滑优化策略，所有模型的最优方案精度损失均实现大幅收窄：ResNet50、RegNetY-16GF 的最优损失控制在 **2个百分点以内**；ViT-B/16 优化收益最显著，从基准13.6个百分点的损失直接压缩至1.25个百分点，完全抹平了与perchannel量化的基础精度差距。同时SmoothO3在perlayer量化中增益突出，8个测试模型里有4个的最优配置均开启了该优化项。

模型名称	SmoothO3	量化粒度	特征校准算法	权重量化算法	量化后Top1精度	浮点基准Top1精度	精度损失率	备注
EfficientNet-B7	❌	perlayer	normal	normal	0.6770	0.7300	-5.30%	基础量化方案
	❌	perlayer	percentile	lsq	**0.7150**	0.7300	-1.50%	精度最优配置
MNASNet1_3	❌	perlayer	normal	normal	0.7150	0.7590	-4.40%	基础量化方案
	✅	perlayer	omse	lsq	**0.7380**	0.7590	-2.10%	精度最优配置
MobileNetV2	❌	perlayer	normal	normal	0.6680	0.7215	-5.35%	基础量化方案
	❌	perlayer	percentile	lsq	**0.6800**	0.7215	-4.15%	精度最优配置
RegNetX-16GF	❌	perlayer	normal	normal	0.7500	0.7900	-4.00%	基础量化方案

模型名称	SmoothO3	量化粒度	特征校准算法	权重量化算法	量化后Top1精度	浮点基准Top1精度	精度损失率	备注
	✅	perlayer	histogram	normal	**0.7715**	0.7900	-1.85%	精度最优配置
RegNetY-16GF	❌	perlayer	normal	normal	0.7605	0.7900	-2.95%	基础量化方案
	❌	perlayer	histogram	normal	**0.7785**	0.7900	-1.15%	精度最优配置
ResNet50	❌	perlayer	normal	normal	0.7815	0.7995	-1.80%	基础量化方案
	✅	perlayer	histogram	normal	**0.7940**	0.7995	-0.55%	精度最优配置
ShuffleNetV2	❌	perlayer	normal	normal	0.6220	0.7600	-13.80%	基础量化方案
	✅	perlayer	histogram	lsq	**0.6665**	0.7600	-9.35%	精度最优配置
ViT-B/16	❌	perlayer	normal	normal	0.6620	0.7980	-13.60%	基础量化方案

模型名称	SmoothO3	量化粒度	特征校准算法	权重量化算法	量化后Top1精度	浮点基准Top1精度	精度损失率	备注
	❌	perlayer	percentile	lsq	**0.7855**	0.7980	-1.25%	精度最优配置

4 混合量化指导

本章节内容适用于文档覆盖的**所有 Amlogic NPU 平台。**

4.1 混合量化简介

`amlnn-toolkit` 工具提供多种混合精度量化方案，可在推理性能与模型精度之间取得平衡，具体支持的场景包括：

- **INT8 + INT16 混合整数量化**：全整型组合，兼顾性能与精度折中。
- **INT8 + Float 混合量化**：敏感层保留浮点精度，其余层走 INT8 加速。
- **INT16 + Float 混合量化**：适合对激活值动态范围要求较高的模型。
- **INT8 + INT16 + Float 混合量化**：三级精度组合，精度优先场景的兜底方案。

混合量化的配置可以基于算子名称和算子类别两种方式配置。

- **基于算子类别配置**：无需预先导出量化 TFLite 模型，直接编辑 `quant_config.json` 文件完成配置。
- **基于算子名称配置**：需先导出量化 TFLite 模型，再参照模型中的算子名称（Output Name）逐个指定混合量化对象。

> 模型转换阶段已支持多种组合的混合量化配置。需要注意的是，在最终生成 ADLA 模型以及 ADLA 模型运行的过程中，部分场景仍可能存在异常，相关能力有待进一步验证与完善。

4.2 混合量化指导

通过结合 8 位与 16 位整数表示，在降低存储与计算开销的同时，保留关键算子的高精度特性。

Step1:导出量化tflite模型

若采用基于算子名称的混合量化配置方式，需预先导出量化 TFLite 模型。导出时在配置中设置 `config(export_intermediate=True)`，以生成包含中间层信息的量化 TFLite 模型，供后续按算子名称定位配置对象。

Step2:混合量化配置说明

- **INT8量化基础配置文件**:

  `amlnn-toolkit` 进行int8转换时，默认的json配置文件如下：

```
[
  {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 8,
        "symmetric": false,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
```

```
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
]
```

- **INT16量化基础配置文件**

```
[
  {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
]
```

- **基于算子类型（Operation Type）的批量配置**

  适用于需要对某一类通用算子统一提升精度的场景。

  - **配置依据**：使用算子的操作类型（`operation`，如 `Sub`、`ADD`、`Conv2D` 等）作为标识。通过operation配置
  - **基于算子类型的INT16批量配置**

    ```
    {
      "regex": ".*",
      "operation": "SUB",
      "algorithm_key": "min_max_uniform_quantize",
      "op_config": {
    ```

```
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
```

- **基于算子类型的Float批量配置**

```
  {
    "regex": ".*",
    "operation": "DIV",
    "algorithm_key": "no_quantize"
  }
```

- **基于算子名称（Output Name）的配置**
  - **配置依据**：使用算子的输出节点名称（`outputname`）作为唯一标识。通过regex参数配置
  - **基于算子名称（Output Name）的INT16配置**

```
{
  "regex": "PartitionedCall_1:0",
  "operation": "*",
  "algorithm_key": "min_max_uniform_quantize",
  "op_config": {
    "activation_tensor_config": {
      "num_bits": 16,
      "symmetric": true,
      "granularity": "TENSORWISE",
      "dtype": "INT",
      "block_size": 0
    },
    "weight_tensor_config": {
      "num_bits": 8,
      "symmetric": true,
      "granularity": "CHANNELWISE",
      "dtype": "INT",
      "block_size": 0
    },
    "compute_precision": "INTEGER",
    "explicit_dequantize": false,
```

```
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
```

- **基于算子名称（Output Name）的Float配置**

```
  {
    "regex":  "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "no_quantize"
  }
```

Step3：编辑混合量化配置文件

- **INT8+INT16混合量化配置文件**

  分别使用基于算子名称与基于算子类别的配置方式后，生成的 `quant_config.json` 配置文件内容如下。

```
[
  {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 8,
        "symmetric": false,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
   {
    "regex": ".*",
    "operation": "SUB",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
```

```
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
  {
    "regex": "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
]
```


- **INT8+Float混合量化配置文件**

  分别使用基于算子名称与基于算子类别的配置方式后，生成的 `quant_config.json` 配置文件内容如下。


```
[
  {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
```

```
        "num_bits": 8,
        "symmetric": false,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
  {
    "regex": ".*",
    "operation": "DIV",
    "algorithm_key": "no_quantize"
   },
   {
    "regex":  "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "no_quantize"
  }
]
```

- **INT16+Float混合量化配置文件**

  分别使用基于算子名称与基于算子类别的配置方式后，生成的 `quant_config.json` 配置文件内容如下。

```
[
   {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
```

```
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
  {
    "regex": ".*",
    "operation": "DIV",
    "algorithm_key": "no_quantize"
   },
   {
    "regex":  "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "no_quantize"
  }
]
```

Step4：执行混合量化

执行转换前，需先通过环境变量指定已修改的配置文件路径，使工具加载自定义量化配置：

```
export AMLNN_SET_QUANT_CONFIG_FILE='Your_dir/quant_config.json'
```

配置完成后，再执行转换脚本即可生效。

5 quantize_optimization_tool 使用指导

本章节内容适用于本文档覆盖的**全部 Amlogic NPU 平台。**`quantize_optimization_tool` 是专为 W8A8 量化场景设计的参数自动搜索工具，其核心机制为遍历激活权重量化算法及其超参数空间，逐一计算各组参数配置下导出的量化 TFLite 模型与浮点模型之间的输出相似度，最终辅助用户定位精度损失最小（即相似度最高）的最优参数组合。

下载链接：https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/function_tool/03_quantize_optimization_tool

5.1 工具介绍

- **工具参数配置如下：**

```
def __init__(self):
    self.model_type='pt2'
    self.model_path='/xxxxx/resnet50_exported.pt2'
    self.source_file='/xxxxxx/ILSVRC2012_val_256.txt'
    self.normalization_mean = [[123.675, 116.28, 103.53]]
    self.normalization_std =  [[58.395, 57.12, 57.375]]
    self.iteration=32
    self.outdir='./'
    self.input_shape=[224,224,3]
    self.input_npy_path=['/xxxx/resnet50_nhwc.npy']
    self.output_npy_path=['/xxxx/resnet50_out_0.npy']
    self.activation_quant_algo=["normal",'percentile',"histogram","omse"]
    self.weight_quant_algo =["normal","lsq"]
    self.inference_input_type="float32"
    self.inference_output_type="float32"
    self.percentile_sigma =[0.02,0.04,0.06,0.08,0.1,0.12,0.14,0.16,0.18,0.2]
    self.percentile_alpha =[0.9999,0.99999,0.999999,0.9999999]
    self.omse_range =[80,85,90,95,98]
    self.omse_step =[0.01,0.012,0.014,0.016,0.018,0.02]
    self.num_histogram_bins = [2048, 4096, 8192 , 12288,16384]
    os.environ['AMLNN_ENABLE_CHANNEL_LAST_IO']=str(True)
    self.result="test_result.txt"
    self.output=[]
    self.euclidean_dis=[]
    self.cos_sim=[]
```

- **参数说明：**
  - **基础配置项**
    - `model_type`：模型格式选择，支持 `pt2`、`tflite`；选择 `pt2` 时工具会自动完成前置格式转换
    - `model_path`：浮点模型的本地路径，根据 `model_type` 对应填写 pt2 模型或 tflite 文件路径
    - `input_shape`：明确指定模型输入张量的维度结构
    - `outdir`：量化模型、搜索日志、结果文件的统一输出目录
    - `iteration`：从校准集中读取的实际量化校准样本数
  - **数据配置项**
    - `source_file`：校准集索引文件路径，即提前准备好的存放图片/npy路径的txt文件

        - `normalization_mean`、`normalization_std`：输入校准集的归一化均值与方差，需与原浮点模型预处理逻辑完全对齐
        - `input_npy_path`、`output_npy_path`：浮点模型参考输入、输出的npy文件路径列表，用于计算相似度
    - **量化搜索配置项**
        - `activation_quant_algo`：激活值量化算法可选范围，支持 `normal`、`percentile`、`histogram`、`omse`
        - `weight_quant_algo`：权重量化算法可选范围，支持 `normal`、`lsq`
        - `percentile_sigma`、`percentile_alpha`：percentile 算法的超参数搜索区间
        - `omse_range`、`omse_step`：omse 算法的超参数范围与遍历步长
        - `num_histogram_bins`：直方图算法的分箱数，常规模型默认设为2048，也可设置为2048的倍数扩展搜索范围
    - **特殊适配项**
        - `inference_input_type`、`inference_output_type`：指定推理阶段输入、输出是否保留浮点类型，可按需关闭首尾层量化
        - 若模型输入为 NCHW 格式，需在脚本开头添加环境变量配置：`os.environ['AMLNN_ENABLE_CHANNEL_LAST_IO'] = 'false'`

5.2 使用指导

- **前期准备**：使用前需提前准备三类核心资源，避免搜索流程中断：
    - **校准数据集索引文件**：`.txt` 格式，每行填写单张图片或单条 `.npy` 数据文件路径
    - **模型预处理参数**：输入校准集对应的归一化均值、归一化标准差
    - **浮点模型参考数据**：用于精度比对的输入 `.npy` 列表、对应输出 `.npy` 列表
- **工具搜索流程**：无需手动逐组调试验证，工具将自动完成全链路操作：
    - 按预设范围遍历所有激活值量化算法，同步遍历权重量化的 `normal`、`lsq` 两种模式
    - 对每组量化配置生成对应量化模型，计算其与浮点参考模型的输出相似度
    - 所有搜索结果按相似度排序，统一写入 `test_result.txt` 输出目录，直接选取排名第一的参数即可作为最终部署方案

5.3 使用建议

1. 初始测试建议先缩小超参数搜索范围，快速跑完验证流程，确认结果无误后再放开全量搜索
2. 校准数据集需覆盖模型实际业务的典型输入场景，否则会出现“相似度高但线上精度差”的偏差
3. 搜索完成后优先选取 `test_result.txt` 中相似度Top1的配置，仅在特定算子掉点时再手动微调局部参数

5.4 自定义评估标准

当前工具内置的评估标准为单帧数据余弦相似度：在参数搜索过程中遍历所有参数组合，以余弦相似度最高的一组作为最终量化参数。然而对于部分模型而言，单帧相似度无法准确体现真实精度水平。为此需引入客户自定义评估标准能力，由客户指定精度评价指标，工具再据此标准完成参数搜索与最优值选取。

- **当前内置评估标准**
    - 参考quantize_optimization_tool.py中的test_result接口

```
def test_result(self, cmd_line):
    self.run_tflite()  # 运行导出的量化 TFLite 模型

    self.compute_euclidean_dis_and_cos()  # 对比量化 TFLite 结果与浮点模型结果

    self.record_result(cmd_line)  # 记录本次转换参数及量化/浮点结果的对比数据
```

- **修改指导**
  - **run_tflite**：在该方法内实现自定义评估标准，运行量化tflite模型并输出指定数据集下的精度结果。
  - **record_result**：记录本次转换所使用的参数配置及对应的精度结果，便于后续参数对比与溯源。

    ```
    def test_result(self, cmd_line):
        self.run_tflite()  # 运行导出的量化 TFLite 模型，并在指定数据集上按自定义评估标准（如 Top1 精度、mAP 等）计算精度指标

        self.record_result(cmd_line)  # 记录本次转换参数及对应的精度结果
    ```

6 hybrid_ratio_tool使用指导

本章节仅适用于本文档覆盖的**Amlogic ADLA3平台**。专为 w4a16、w4a8 量化场景设计，核心目标是在低比特压缩率与模型精度之间找到最优平衡点。工具通过 `hybrid_ratio` 参数精准控制 4bit 量化 Conv2D 算子的占比

- 取值范围为 0~1：`hybrid_ratio = 0` 时，所有 Conv2D 算子全部执行 INT8 量化；`hybrid_ratio = 1` 时，所有 Conv2D 算子全部执行 INT4 量化。
- 工具会自动遍历预设的 `hybrid_ratio` 搜索区间，同步输出每组配置下量化模型与浮点模型的余弦相似度、欧式距离两项核心精度指标，用户可结合业务对精度、性能的双重要求，自主选定最终的最优量化参数组合。

下载链接：https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/function_tool/04_hybrid_ratio_tool

6.1 工具介绍

工具参数配置如下：

```
def __init__(self):

    self.model_type='pt2' #[pt2/tflite]
    self.trunk_name ="cat_16"
    self.model_path='/xxxx/yolovx-s.pt2'
    self.source_file='/xxxx/dataset.txt'
    self.normalization_mean = [[0.,0.,0.]]
    self.normalization_std =  [[255.0,255.0,255.0]]
    self.quantized_dtype="w4a16" #["w4a8","w4a16"]
    self.hybrid_ratio_all =np.round(np.linspace(0.1,0.5,2),2)
    self.groups=[32]
    self.iteration=32
    self.outdir='./'
    self.input_npy_path=['yolov8n_trunc_nhwc.npy']
    self.output_npy_path=['yolov8n_trunc_nhwc_out_0.npy']
    self.activation_quant_algo=["normal",'percentile',"histogram","omse"]
    self.weight_quant_algo =["lsq","decoupleQ"]
    self.inference_input_type="float32"
    self.inference_output_type="float32"

    self.percentile_sigma =[0.02,0.04,0.06,0.08,0.1,0.12,0.14,0.16,0.18,0.2]
    self.percentile_alpha =[0.9999,0.99999,0.999999,0.9999999]
    self.omse_range =[80,85,90,95,98]
    self.omse_step =[0.01,0.012,0.014,0.016,0.018,0.02]
    self.num_histogram_bins = [2048, 4096, 8192 , 12288,16384]
    os.environ['AMLNN_ENABLE_CHANNEL_LAST_IO']=str(True)
    self.result="test_result.txt"
    self.output=[]
    self.euclidean_dis=[]
    self.cos_sim=[]
```

参数说明：

- **model_type**：支持 `pt2`、`tflite` 两种模型输入格式；若输入为 pt2 模型，可通过 `trunk_name` 指定输出节点名称，对模型执行截断处理。
- **quantized_dtype**：量化精度模式，可选 `w4a16`、`w4a8`。w4a8 场景下权重量化默认启用 lsq 算法，无需额外配置；w4a16 场景下支持手动选择 lsq、decoupleQ两种权重量化算法。
- **hybrid_ratio_all**：混合量化比例的遍历搜索区间，取值范围限定为 0~1，工具将在该区间内自动生成多组比例组合完成全量验证。
- **groups**：全连接（fc）算子 block_wise 分组量化的分组数，支持配置为 `[32, 64, 128]`；开启该配置时，fc 算子的激活值将跳过量化，保持高精度。
- **weight_quant_algo**：仅在 w4a16 量化场景下生效，用于指定权重量化算法，可选值为 `lsq`、`decoupleQ`。
- **activation_quant_algo**：仅在 w4a8 量化场景下生效，支持遍历激活值量化算法的完整参数空间，自动锁定最优激活量化配置。

6.2 YOLOX-S 模型验证

6.2.1 W4A16 量化验证

本实验以YOLOX-S浮点模型（mAP50=0.593、mAP50-95=0.405）为基准，验证w4a16场景下decoupleQ与LSQ两种权重量化算法在不同混合精度下的精度表现，核心通过`hybrid_ratio`参数（0.0=全INT8量化，1.0=全INT4量化）调控INT4卷积层占比(特征不量化)，数值越高低比特量化占比越大。

- **验证结果**

hybrid_ratio	weight_quant_algo	map50	map50_95
0.0	decoupleQ	0.592	0.404
0.0	lsq	0.559	0.376
0.1	decoupleQ	0.591	0.401
0.1	lsq	0.548	0.362
0.2	decoupleQ	0.586	0.395
0.2	lsq	0.541	0.352
0.3	decoupleQ	0.582	0.389
0.3	lsq	0.528	0.337
0.4	decoupleQ	0.58	0.383
0.4	lsq	0.524	0.323
0.5	decoupleQ	0.577	0.377
0.5	lsq	0.512	0.308
0.6	decoupleQ	0.574	0.374
0.6	lsq	0.501	0.299
0.7	decoupleQ	0.563	0.366

hybrid_ratio	weight_quant_algo	map50	map50_95
0.7	lsq	0.486	0.291
0.8	decoupleQ	0.561	0.363
0.8	lsq	0.48	0.283
0.9	decoupleQ	0.54	0.347
0.9	lsq	0.434	0.252
1.0	decoupleQ	0.532	0.331
1.0	lsq	0.446	0.254

- **结果分析：**
  - **全INT8量化（hybrid_ratio=0.0）**：decoupleQ精度几乎与浮点模型持平，mAP50、mAP50-95仅各降0.001，几乎无损失；LSQ两项指标分别下降0.034、0.029，已出现明显精度衰减
  - **中低混合量化区间（0.1~0.5）**：decoupleQ精度平缓均匀下降，hybrid_ratio=0.5时仅小幅损失；LSQ衰减速度持续加快，同等配比下精度损失远高于decoupleQ
  - **高比例INT4区间（0.6~1.0）**：精度差距进一步拉大，全INT4状态下decoupleQ mAP50=0.532、mAP50-95=0.331，仍保持可观检测能力；LSQ精度大幅跳水，mAP50仅0.446、mAP50-95低至0.254

6.2.2 w4a8量化验证

本实验以YOLOX-S浮点模型（mAP50=0.593、mAP50-95=0.405）为基准， 测试W4A8量化场景下不同`hybrid_ratio`配比下两种权重量化算法的精度表现。

- **验证结果**

hybrid_ratio	weight_quant_algo	map50	map50_95
0.0	decoupleQ	0.58	0.387
0.0	lsq	0.404	0.255
0.1	decoupleQ	0.579	0.387
0.1	lsq	0.383	0.238
0.2	decoupleQ	0.578	0.385
0.2	lsq	0.377	0.233
0.3	decoupleQ	0.576	0.383
0.3	lsq	0.36	0.218
0.4	decoupleQ	0.573	0.377
0.4	lsq	0.356	0.209
0.5	decoupleQ	0.572	0.374

hybrid_ratio	weight_quant_algo	map50	map50_95
0.5	lsq	0.346	0.201
0.6	decoupleQ	0.57	0.372
0.6	lsq	0.335	0.194
0.7	decoupleQ	0.567	0.37
0.7	lsq	0.32	0.185
0.8	decoupleQ	0.564	0.368
0.8	lsq	0.324	0.188
0.9	decoupleQ	0.559	0.365
0.9	lsq	0.285	0.163
1.0	decoupleQ	0.556	0.354
1.0	lsq	0.462	0.268

- **结果分析：**
  - **整体规律**：随着`hybrid_ratio`升高、4bit权重卷积层占比提升，两种算法的mAP50、mAP50-95精度均逐步下降，且全量化配比下**decoupleQ的所有精度指标均高于LSQ。**
  - 纯8bit权重量化（`hybrid_ratio=0.0`）：decoupleQ的mAP50=0.58、mAP50-95=0.387，整体精度损失极小。
  - 全4bit权重量化（`hybrid_ratio=1.0`）：decoupleQ仍保持可观可用精度，mAP50=0.556、mAP50-95=0.354。

6.3 使用建议

- **适用场景**：在 W4A8 或 W4A16 量化部署中，若全 INT4 权重量化（`hybrid_ratio=1.0`）导致精度不达标，请启用本工具进行混合精度调优。
- **操作策略**：利用工具遍历 `hybrid_ratio` 搜索空间，自动评估不同混合比例下的精度表现，从而在模型压缩率与推理精度之间锁定最优平衡点。
- **参数选型：**
  - **Weight Quant Algo**：强烈建议优先选用 **decoupleQ** 算法，因其在低比特量化下具备更优异的精度保持能力。
  - **Hybrid Ratio**：根据工具输出的精度-压缩比曲线，选取满足业务精度阈值的最小 INT4 占比作为最终转换参数。